Web Application Firewall Home Lab using SafeLine WAF

1. Introduction

This lab documents the setup of a complete cybersecurity home lab using VirtualBox, Kali Linux, Ubuntu Server, and SafeLine WAF. The purpose of this lab is to demonstrate how a Web Application Firewall (WAF) can protect a vulnerable web application against common web attacks.

The lab objectives were:

- Set up a vulnerable web application (DVWA) on Ubuntu Server
- Demonstrate how to perform SQL injection, XSS, and command injection attacks from Kali Linux
- Show how SafeLine WAF detects, logs, and blocks such attacks
- Explore advanced WAF features including HTTP flood defense, custom deny rules, and authentication gateway

Ubuntu ip

```
halima@halima-VirtualBox:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::e72d:e321:1f2f:849f  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:d9:57:f7  txqueuelen 1000  (Ethernet)
        RX packets 247  bytes 221618 (221.6 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 286  bytes 22403 (22.4 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 206  bytes 16914 (16.9 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 206  bytes 16914 (16.9 KB)
```

kali ip

```
(base) ┌──(kali㉿kali)-[~]
└─$ ifconfig
br-316095af1220: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.18.0.1  netmask 255.255.0.0  broadcast 172.18.255.255
        ether 02:42:d9:4f:61:07  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:4b:7b:a8:fe  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 6 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.5  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::9b84:60f2:c0cf:c55f  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:63:b0:05  txqueuelen 1000  (Ethernet)
        RX packets 50  bytes 10588 (10.3 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 47  bytes 6128 (5.9 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.217.7  netmask 255.255.255.0  broadcast 192.168.217.255
        inet6 fe80::bd9e:b62e:5406:6463  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:38:d7:60  txqueuelen 1000  (Ethernet)
        RX packets 18  bytes 4790 (4.6 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 47  bytes 6128 (5.9 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 480 (480.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 480 (480.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Your confirmed IP map for this entire lab:

VM	IP	Role
Kali	10.0.2.5	Attacker
Ubuntu	10.0.2.15	Target server

2. Lab Environment

2.1 Network Configuration

Both virtual machines were configured using NAT Network in VirtualBox, placing them on the same isolated subnet (10.0.2.0/24) while still allowing internet access through the host machine. This approach is safer than Bridged networking as it keeps the intentionally vulnerable DVWA application isolated from the local home network.

2.2 Technology Stack

Component	Status / Details
Hypervisor	Oracle VirtualBox
Attacker VM	Kali Linux (10.0.2.5)
Target VM	Ubuntu Server 22.04 LTS (10.0.2.15)

Web Stack	Apache2, PHP 8.1, MySQL 8.0 (LAMP)
Vulnerable App	DVWA (Damn Vulnerable Web Application)
WAF	SafeLine WAF v9.3.6 (Docker-based)
Networking	VirtualBox NAT Network
SSL	Self-signed certificate for dvwa.local

3. Lab Setup Steps

Step 1 - Verify Network Connectivity

After configuring both VMs on the NAT Network, connectivity was verified by running ifconfig on each VM to confirm their IP addresses, then pinging Ubuntu from Kali.

```
(base) ┌──(kali㉿kali)-[~]
└─$ ping 10.0.2.15
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=3.04 ms
64 bytes from 10.0.2.15: icmp_seq=2 ttl=64 time=0.889 ms
64 bytes from 10.0.2.15: icmp_seq=3 ttl=64 time=1.69 ms
64 bytes from 10.0.2.15: icmp_seq=4 ttl=64 time=1.98 ms
64 bytes from 10.0.2.15: icmp_seq=5 ttl=64 time=2.64 ms
64 bytes from 10.0.2.15: icmp_seq=6 ttl=64 time=2.06 ms
64 bytes from 10.0.2.15: icmp_seq=7 ttl=64 time=1.43 ms
64 bytes from 10.0.2.15: icmp_seq=8 ttl=64 time=1.70 ms
64 bytes from 10.0.2.15: icmp_seq=9 ttl=64 time=1.47 ms
64 bytes from 10.0.2.15: icmp_seq=10 ttl=64 time=2.80 ms
64 bytes from 10.0.2.15: icmp_seq=11 ttl=64 time=2.24 ms
64 bytes from 10.0.2.15: icmp_seq=12 ttl=64 time=1.61 ms
64 bytes from 10.0.2.15: icmp_seq=13 ttl=64 time=1.15 ms
64 bytes from 10.0.2.15: icmp_seq=14 ttl=64 time=2.68 ms
```

Result: Kali successfully reached Ubuntu with consistent response times of 1-3ms, confirming NAT Network routing was working correctly.

Step 2 - Ubuntu Initial Setup

The Ubuntu Server was updated and required tools were installed:

sudo apt-get update
sudo apt-get upgrade -y
sudo apt-get install -y net-tools
sudo apt-get install -y openssl

```
halima@halima-VirtualBox:~$ sudo apt-get install -y net-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
net-tools is already the newest version (1.60+git20181103.0eebece-1ubuntu5.4).
The following packages were automatically installed and are no longer required:
  libwpe-1.0-1 libwpebackend-fdo-1.0-1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded.
halima@halima-VirtualBox:~$ sudo apt-get install -y openssl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openssl is already the newest version (3.0.2-0ubuntu1.23).
openssl set to manually installed.
The following packages were automatically installed and are no longer required:
  libwpe-1.0-1 libwpebackend-fdo-1.0-1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded.
halima@halima-VirtualBox:~$
```

Step 3 - Install LAMP Stack

Apache2, PHP, MySQL, and Git were installed as the foundation for hosting DVW

sudo apt-get install -y apache2 php php-mysql mysql-server git

```
halima@halima-VirtualBox:~$ sudo apt-get install -y apache2 php php-mysql mysql-server git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libwpe-1.0-1 libwpebackend-fdo-1.0-1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils git-man libaio1 libapache2-mod-php8.1 libapr1 libaprutil1 libaprutil1-dbd-sqlite3 libaprutil1-ldap libcgi-fast-perl libcgi-pm-perl liberror-perl
  libevent-core-2.1-7 libevent-pthreads-2.1-7 libfcgi-bin libfcgi-perl libfcgi0ldbl libhtml-template-perl liblua5.3-0 libmecab2 libprotobuf-lite23 mecab-ipadic mecab-ipadic-utf8 mecab-utils
  mysql-client-8.0 mysql-client-core-8.0 mysql-common mysql-server-8.0 mysql-server-core-8.0 php-common php8.1 php8.1-cli php8.1-common php8.1-mysql php8.1-opcache php8.1-readline
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn php-pear libipc-sharedcache-perl
  mailx tinyca
The following NEW packages will be installed:
  apache2 apache2-bin apache2-data apache2-utils git git-man libaio1 libapache2-mod-php8.1 libapr1 libaprutil1 libaprutil1-dbd-sqlite3 libaprutil1-ldap libcgi-fast-perl libcgi-pm-perl liberror-perl
  libevent-core-2.1-7 libevent-pthreads-2.1-7 libfcgi-bin libfcgi-perl libfcgi0ldbl libhtml-template-perl liblua5.3-0 libmecab2 libprotobuf-lite23 mecab-ipadic mecab-ipadic-utf8 mecab-utils
  mysql-client-8.0 mysql-client-core-8.0 mysql-common mysql-server mysql-server-8.0 mysql-server-core-8.0 php php-common php-mysql php8.1 php8.1-cli php8.1-common php8.1-mysql php8.1-opcache
  php8.1-readline
0 upgraded, 42 newly installed, 0 to remove and 9 not upgraded.
Need to get 41.0 MB of archives.
After this operation, 294 MB of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libapr1 amd64 1.7.0-8ubuntu0.22.04.2 [108 kB]
```

MySQL was then secured using the secure installation script, removing anonymous users, disabling remote root login, and removing the test database.

sudo mysql_secure_installation

```
halima@halima-VirtualBox:~$ sudo mysql_secure_installation

Securing the MySQL server deployment.

Connecting to MySQL using a blank password.

VALIDATE PASSWORD COMPONENT can be used to test passwords
and improve security. It checks the strength of password
and allows the users to set only those passwords which are
secure enough. Would you like to setup VALIDATE PASSWORD component?

Trash |Y for Yes, any other key for No: N

Skipping password set for root as authentication with auth_socket is used by default.
If you would like to use password authentication instead, this can be done with the "ALTER_USER"
 command.
See https://dev.mysql.com/doc/refman/8.0/en/alter-user.html#alter-user-password-management for m
ore information.

By default, a MySQL installation has an anonymous user,
allowing anyone to log into MySQL without having to have
a user account created for them. This is intended only for
testing, and to make the installation go a bit smoother.
You should remove them before moving into a production
environment.

Remove anonymous users? (Press y|Y for Yes, any other key for No) : Y
```

```
Normally, root should only be allowed to connect from
'localhost'. This ensures that someone cannot guess at
the root password from the network.

Disallow root login remotely? (Press y|Y for Yes, any other key for No) : Y
Success.

By default, MySQL comes with a database named 'test' that
anyone can access. This is also intended only for testing,
and should be removed before moving into a production
environment.


Remove test database and access to it? (Press y|Y for Yes, any other key for No) : Y
 - Dropping test database...
Success.

 - Removing privileges on test database...
Success.

Reloading the privilege tables will ensure that all changes
made so far will take effect immediately.

Reload privilege tables now? (Press y|Y for Yes, any other key for No) : Y
Success.

All done!
```

Once that's done, verify Apache is running:

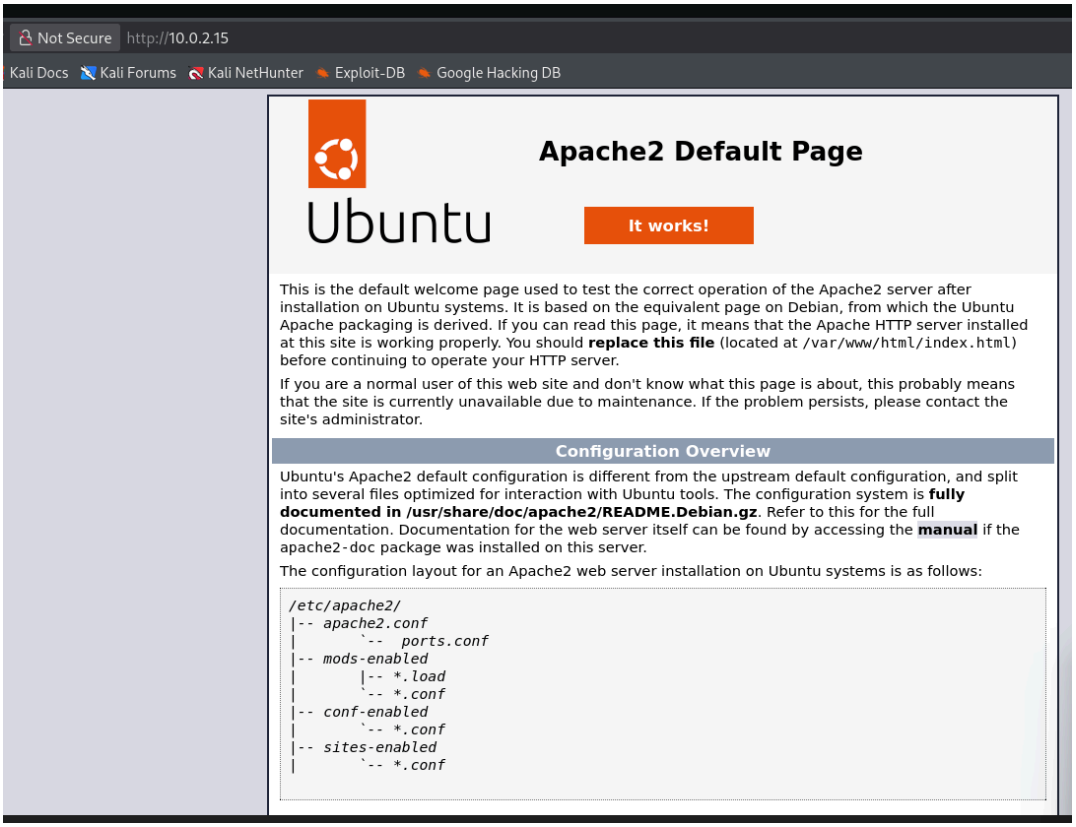
sudo systemctl status apache2

```
halima@halima-VirtualBox:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2026-04-26 10:42:46 EDT; 2min 28s ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 10956 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 10961 (apache2)
      Tasks: 6 (limit: 4880)
     Memory: 10.4M
        CPU: 73ms
     CGroup: /system.slice/apache2.service
             ├─10961 /usr/sbin/apache2 -k start
             ├─10963 /usr/sbin/apache2 -k start
             ├─10964 /usr/sbin/apache2 -k start
             ├─10965 /usr/sbin/apache2 -k start
             ├─10966 /usr/sbin/apache2 -k start
             └─10967 /usr/sbin/apache2 -k start

Apr 26 10:42:46 halima-VirtualBox systemd[1]: Starting The Apache HTTP Server...
Apr 26 10:42:46 halima-VirtualBox apachectl[10960]: AH00558: apache2: Could not reliably determ
Apr 26 10:42:46 halima-VirtualBox systemd[1]: Started The Apache HTTP Server.
```

Then from Kali, open a browser and go to: **http://10.0.2.15**

You should see the default Apache2 Ubuntu Default Page: that confirms Apache is working and Kali can reach Ubuntu's web server.



Step 4 - Install and Configure DVWA

DVWA was cloned from GitHub into Apache's web root and configured to connect to a dedicated MySQL database:On Ubuntu, run:

cd /var/www/html

sudo git clone https://github.com/digininja/DVWA.git

```
halima@halima-VirtualBox:~$ cd /var/www/html
halima@halima-VirtualBox:/var/www/html$ sudo git clone https://github.com/digininja/DVWA.git
[sudo] password for halima:
Cloning into 'DVWA'...
remote: Enumerating objects: 5703, done.
remote: Counting objects: 100% (35/35), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 5703 (delta 25), reused 17 (delta 17), pack-reused 5668 (from 2)
Receiving objects: 100% (5703/5703), 2.72 MiB | 1.93 MiB/s, done.
Resolving deltas: 100% (2842/2842), done.
halima@halima-VirtualBox:/var/www/html$
```

Once cloned, set permissions:

sudo chown -R www-data:www-data DVWA

sudo chmod -R 755 DVWA

Now copy and edit the config file:

sudo cp /var/www/html/DVWA/config/config.inc.php.dist /var/www/html/DVWA/config/config.inc.php

sudo nano /var/www/html/DVWA/config/config.inc.php

Inside the file, find these lines and make sure they match exactly:

$_DVWA['db_database'] = 'dvwa';

$_DVWA['db_user'] = 'dvwa_user';

$_DVWA['db_password'] = 'p@ssw0rd';

Press Ctrl+X → Y → Enter to save and exit.

A dedicated MySQL database and user were created:

sudo mysql -u root -p

It will ask for password — just press Enter (no password set for root). Then run:

CREATE DATABASE dvwa;

CREATE USER 'dvwa_user'@'localhost' IDENTIFIED BY 'p@ssw0rd';

GRANT ALL ON dvwa.* TO 'dvwa_user'@'localhost';

FLUSH PRIVILEGES;

Exit;

The DVWA config file was updated with the correct database credentials (db_user set to dvwa_user, db_password set to p@ssw0rd).

```
halima@halima-VirtualBox:/var/www/html$ sudo chown -R www-data:www-data DVWA
halima@halima-VirtualBox:/var/www/html$ sudo chmod -R 755 DVWA
halima@halima-VirtualBox:/var/www/html$ sudo cp /var/www/html/DVWA/config/config.inc.php.dist /v
ar/www/html/DVWA/config/config.inc.php
halima@halima-VirtualBox:/var/www/html$ sudo nano /var/www/html/DVWA/config/config.inc.php
halima@halima-VirtualBox:/var/www/html$ sudo nano /var/www/html/DVWA/config/config.inc.php
halima@halima-VirtualBox:/var/www/html$ sudo mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.45-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE dvwa;
Query OK, 1 row affected (0.03 sec)

mysql> CREATE USER 'dvwa_user'@'localhost' IDENTIFIED BY 'p@ssw0rd';
Query OK, 0 rows affected (0.02 sec)

mysql> GRANT ALL ON dvwa.* TO 'dvwa_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.02 sec)

mysql> exit;
Bye
```

Step 5 - Move DVWA to Port 8080

Since SafeLine WAF would take port 80/443, Apache was reconfigured to listen on port 8080. Two files were edited

On **Ubuntu**, run:

sudo nano /etc/apache2/ports.conf

Find this line: Listen 80 change it to: **Listen 8080**

Ctrl+X → Y → Enter to save.

```
  GNU nano 6.2                                  /etc/apa
# If you just change the port or add more ports
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf
Ubuntu Software
Listen 8080

<IfModule ssl_module>
        Listen 443
</IfModule>

<IfModule mod_gnutls.c>
        Listen 443
</IfModule>

# vim: syntax=apache ts=4 sw=4 sts=4 sr noet
```

Now edit the virtual host:

sudo nano /etc/apache2/sites-available/000-default.conf

Find this line: <VirtualHost *:80> change it to: <VirtualHost *:8080>

Ctrl+X → Y → Enter to save.

```
<VirtualHost *:8080>
        # The ServerName directive sets the request scheme, hostname and port that
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
Help    # However, you must set it for any further virtual host explicitly.
        #ServerName www.example.com

        ServerAdmin webmaster@localhost
        DocumentRoot /var/www/html

        # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
        # error, crit, alert, emerg.
        # It is also possible to configure the loglevel for particular
        # modules, e.g.
        #LogLevel info ssl:warn

        ErrorLog ${APACHE_LOG_DIR}/error.log
        CustomLog ${APACHE_LOG_DIR}/access.log combined

        # For most configuration files from conf-available/, which are
        # enabled or disabled at a global level, it is possible to
        # include a line for only one particular virtual host. For example the
        # following line enables the CGI configuration for this host only
        # after it has been globally disabled with "a2disconf".
        #Include conf-available/serve-cgi-bin.conf
</VirtualHost>

# vim: syntax=apache ts=4 sw=4 sts=4 sr noet
```

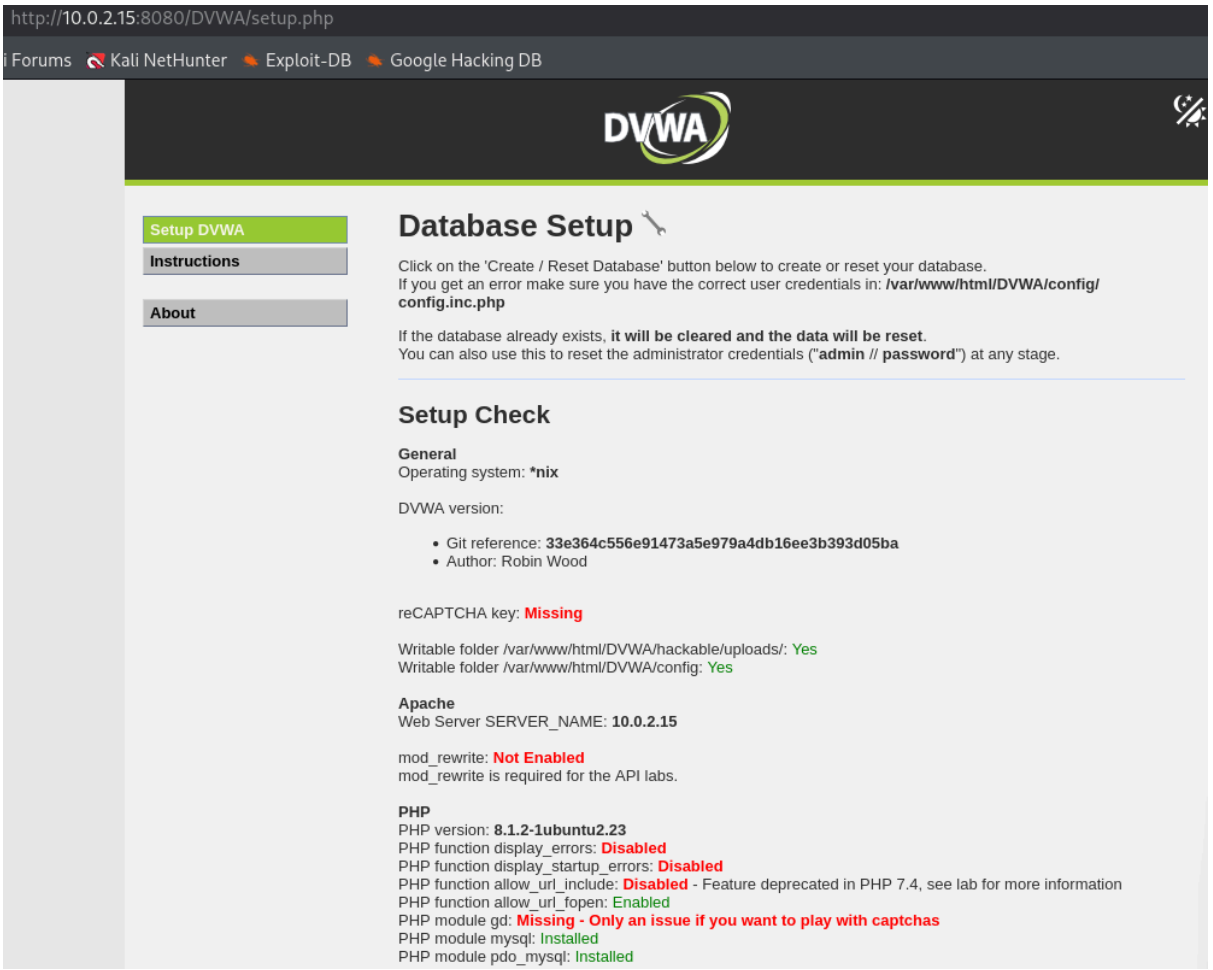
Now restart Apache:

sudo systemctl restart apache2

Verify DVWA is accessible

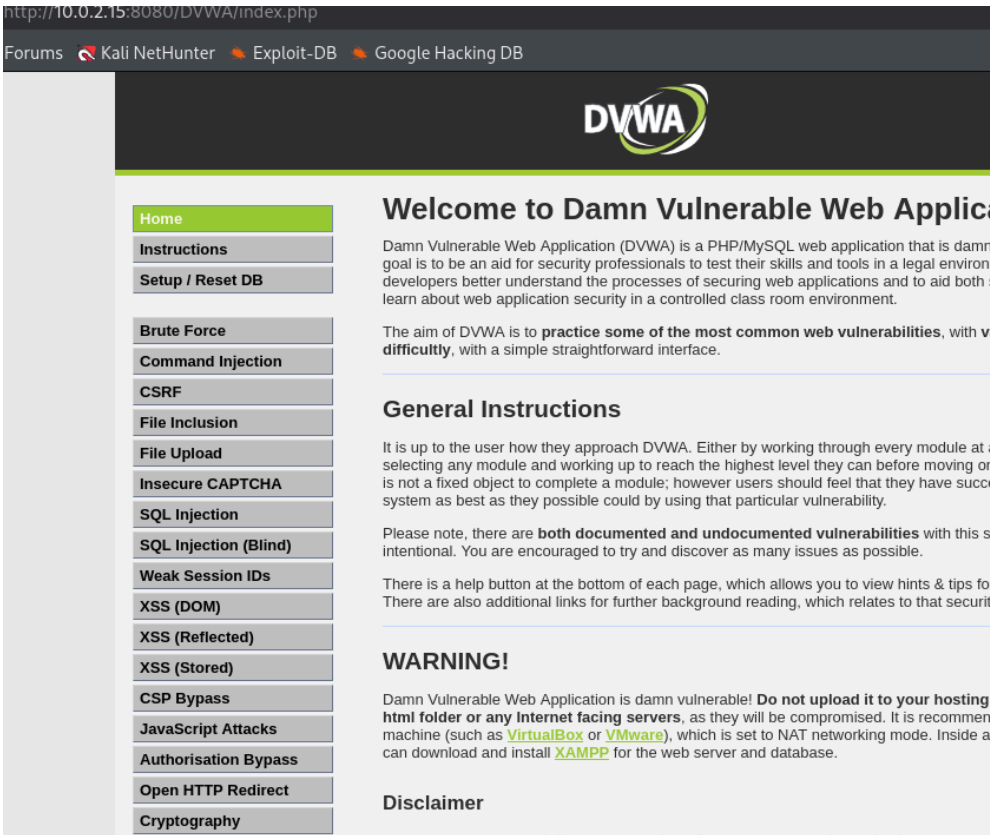
From **Kali browser**, go to:

http://10.0.2.15:8080/DVWA/setup.php



DVWA was then accessible from Kali at: http://10.0.2.15:8080/DVWA/setup.php

The Create / Reset Database button was clicked to initialize DVWA successfully.



Step 6 - Add Custom Database Values

Once logged in, go back to **Ubuntu** and run:

sudo mysql -u root

Then paste all of this:

```
USE dvwa;

CREATE TABLE test_users (

    id INT NOT NULL AUTO_INCREMENT,

    username VARCHAR(50) NOT NULL,

    password VARCHAR(50) NOT NULL,

    PRIMARY KEY (id)

);
```

INSERT INTO test_users (username, password) VALUES

('alice', 'alice123'),

('bob', 'bob123'),

('admin', 'admin123');

exit;

These give us real data to target when we demonstrate SQL injection later.

```
halima@halima-VirtualBox:~$ sudo mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 23
Server version: 8.0.45-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE dvwa;
Reading table information for completion of table and column names
Y[illegible]ff this feature to get a quicker startup with -A
VBox_GAs_7.2.6
Database changed
mysql> CREATE TABLE test_users (
    ->     id INT NOT NULL AUTO_INCREMENT,
    ->     username VARCHAR(50) NOT NULL,
    ->     password VARCHAR(50) NOT NULL,
    ->     PRIMARY KEY (id)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO test_users (username, password) VALUES
    -> ('alice', 'alice123'),
    -> ('bob', 'bob123'),
    -> ('admin', 'admin123');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> exit;
Bye
```

Step 7 - DNS Setup

This lets both VMs use dvwa.local instead of typing the IP every time.

On **Ubuntu**, run:

sudo nano /etc/hosts

Add this line at the bottom:

10.0.2.15 dvwa.local www.dvwa.local

```
  GNU nano 6.2                                    /etc/hosts
127.0.0.1       localhost
127.0.1.1       halima-VirtualBox

Ubuntu Software  lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
10.0.2.15    dvwa.local www.dvwa.local
```

Now do the same on **Kali**:

sudo nano /etc/hosts

Add the same line at the bottom:

10.0.2.15 dvwa.local www.dvwa.local

```
  GNU nano 8.7.1                                  /etc/hos
127.0.0.1       localhost
127.0.1.1       kali
::1             localhost ip6-localhost ip6-loopback
ff02::1         ip6-allnodes
ff02::2         ip6-allrouters
10.48.153.172 delivery.cupidsarrow.thm
10.48.164.118 nova.thm
10.0.2.15    dvwa.local www.dvwa.local
```

Ctrl+X → Y → Enter to save.

Verify DNS is working

On **Kali**, run:

ping dvwa.local

```
(base) ┌──(kali㉿kali)-[~]
└─$ ping dvwa.local
PING dvwa.local (10.0.2.15) 56(84) bytes of data.
64 bytes from dvwa.local (10.0.2.15): icmp_seq=1 ttl=64 time=2.32 ms
64 bytes from dvwa.local (10.0.2.15): icmp_seq=2 ttl=64 time=1.30 ms
64 bytes from dvwa.local (10.0.2.15): icmp_seq=3 ttl=64 time=0.935 ms
64 bytes from dvwa.local (10.0.2.15): icmp_seq=4 ttl=64 time=2.11 ms
```

You should see it resolving to 10.0.2.15 and getting replies. Press Ctrl+C to stop.

Also try from Kali browser:

http://dvwa.local:8080/DVWA

You should see the DVWA login page using the domain name instead of IP.

Step 8 - Create Self-Signed SSL Certificate

On Ubuntu, run:

```
halima@halima-VirtualBox:~$ sudo mkdir /etc/ssl/dvwa
[sudo] password for halima:
halima@halima-VirtualBox:~$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/dvwa/dvwa.key \
  -out /etc/ssl/dvwa/dvwa.crt
...+.+.....+....+..+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.+..+...+
.....++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*....+.......+........+.
..........+...........+.........+.+......+..+..........+......+.....+....+...+.........+......
......+....+..................+...+...........+.+..+......++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++
..+.+..+.............+.....+.+...+..++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
+++++*.+...+.....+.+...+.....++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
..+...+........+.......+.....+....+...+.....+.+.....+.........+...+...+.+.........+.....+.....
+.....+........+.+............+...+..............+....+.........+.....+......+.+.........+...
.+...+...+..............+.....+..........+.........+.....+....+.....+....+......+.........+.
.............+...........+.......+........+.+......+..+.......+........+.......+...........+..
....+.......................+....+...+.....+.+..+.......+..+...............+......+.........+.
......+....+......+...........+..........+.....+...+...+.......+...+..+......+....+..+...+.+..
.+.......+........+....+.....+..........+...+...+..............+..........+........+....+...+.
+........+....+..+.+...+......+......+.....+....+.....+...+...+....+.....+....+...............
.........+....+.........+...+.........+........+......+.+......+..+...+....+...+..............
.+.+.....+.......................+...+...+....+.........+..+.......+..+.+...................
.+....+..+......+...+.+...+...........+.......+..+...+.........+...+.+.........+........+....+
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:ID
State or Province Name (full name) [Some-State]:Jakarta
Locality Name (eg, city) []:Jarkata
Organization Name (eg, company) [Internet Widgits Pty Ltd]:DVWA Lab
Organizational Unit Name (eg, section) []:Security
Common Name (e.g. server FQDN or YOUR name) []:dvwa.local
Email Address []:admin@dvwa.local
halima@halima-VirtualBox:~$
```

Verify the certificate was created:

ls /etc/ssl/dvwa/

You should see two files:

```
halima@halima-VirtualBox:~$ ls /etc/ssl/dvwa/
dvwa.crt  dvwa.key
```

Step 9 - Install SafeLine WAF

On **Ubuntu**, run:

Sudo bash -c "$(curl -fsSLk https://waf.chaitin.com/release/latest/manager.sh)" -- --en

```
halima@halima-VirtualBox:~$ sudo bash -c "$(curl -fsSLk https://waf.chaitin.com/release/latest/manager.sh)" -- --en

[INFO  11:27:09]: SafeLine is a self-hosted WAF(Web Application Firewall) to protect your web apps from attacks and explo
ts.
[INFO  11:27:09]: A web application firewall helps protect web apps by filtering and monitoring HTTP traffic between a wel
application and the Internet. It typically protects web apps from attacks such as SQL injection, XSS, code injection, os
Terminal
injection, CRLF injection, ldap injection, xpath injection, RCE, XXE, SSRF, path traversal, backdoor, bruteforce,
http-flood, bot abused, among others.

Choice what do you want to do  [ 1.INSTALL  2.UPGRADE  3.UNINSTALL  4.REPAIR  5.RESTART ]  (1/2/3/4/5): 1
[INFO  11:27:18]: Will be going to installing SafeLine WAF for you.
[INFO  11:27:18]: Checking docker version
[WARN  11:27:18]: Running SafeLine WAF requires Docker, but Docker is not installed
Do you want the latest version of Docker to be automatically installed for you?  [ y.Yes  n.No ]  (y/n): y
[INFO  11:27:28]: Docker is being installed for you. It will take a few minutes. Please wait patiently.
[INFO  11:29:20]: Checking docker compose version
[INFO  11:29:20]: Installation environment check passed
Input the path to install SafeLine WAF  (Keep blank default to /data/safeline):
```

When it finishes it will show you something like:

```
 Image swr.cn-east-3.myhuaweicloud.com/chaitin-safeline/safeline-luigi-g:9.3.6 [⠿  ] 10.49MB / 79.99MB          Pullin
g                                                                                    112.6s
 Image swr.cn-east-3.myhuaweicloud.com/chaitin-safeline/safeline-fvm-g:9.3.6 [⠿⠿⠿⠿⠿⠿⠿⠿⠿⠿] 47.52MB[+] pull 41/[+[[+] pull 42
[+] pull 71/72                                                    Image swr.cn-east-3.m  I   Image swr.cn-east-3
 ✔ Image swr.cn-east-3.myhuaweicloud.com/chaitin-safeline/safeline-postgres:15.2    Pulled                          412.3s
 ✔ Image swr.cn-east-3.myhuaweicloud.com/chaitin-safeline/safeline-tengine-g:9.3.6  Pulled                          242.4s
 ✔ Image swr.cn-east-3.myhuaweicloud.com/chaitin-safeline/safeline-mgt-g:9.3.6      Pulled                          240.4s
 ✔ Image swr.cn-east-3.myhuaweicloud.com/chaitin-safeline/safeline-luigi-g:9.3.6    Pulled                          404.1s
 ✔ Image swr.cn-east-3.myhuaweicloud.com/chaitin-safeline/safeline-fvm-g:9.3.6      Pulled                          370.7s
 ✔ Image swr.cn-east-3.myhuaweicloud.com/chaitin-safeline/safeline-detector-g:9.3.6 Pulled                          266.1s
 ✔ Image swr.cn-east-3.myhuaweicloud.com/chaitin-safeline/safeline-chaos-g:9.3.6    Pulled                          401.1s
[INFO  11:38:44]: Starting Docker containers
[INFO  11:39:07]: SafeLine WAF installation completed
[INFO  11:39:10]: Wait for mgt healthy
[INFO  11:39:16]: Wait for mgt healthy
[INFO  11:39:21]: Wait for mgt healthy
[INFO  11:39:26]: Wait for mgt healthy
[INFO  11:39:31]: Wait for mgt healthy
[INFO  11:39:37]: Wait for mgt healthy
[INFO  11:39:42]: Wait for mgt healthy
[INFO  11:39:47]: Setup admin
[INFO  11:40:04]:
[INFO] Initial username : admin
[INFO] Initial password : fWQFoG9V
[INFO] Done
[INFO  11:40:04]: SafeLine WAF management panel: https://10.0.2.15:9443/
[INFO  11:40:04]: SafeLine WAF management panel: https://0.0.0.0:9443/

https://discord.gg/SVnZGzHFvn

Join discord group for more informations of SafeLine by above address
```

SafeLine is installed! Here are your credentials:

URL	https://10.0.2.15:9443
Username	admin
Password	fWQFoG9V

While it's installing, here's what SafeLine is doing:

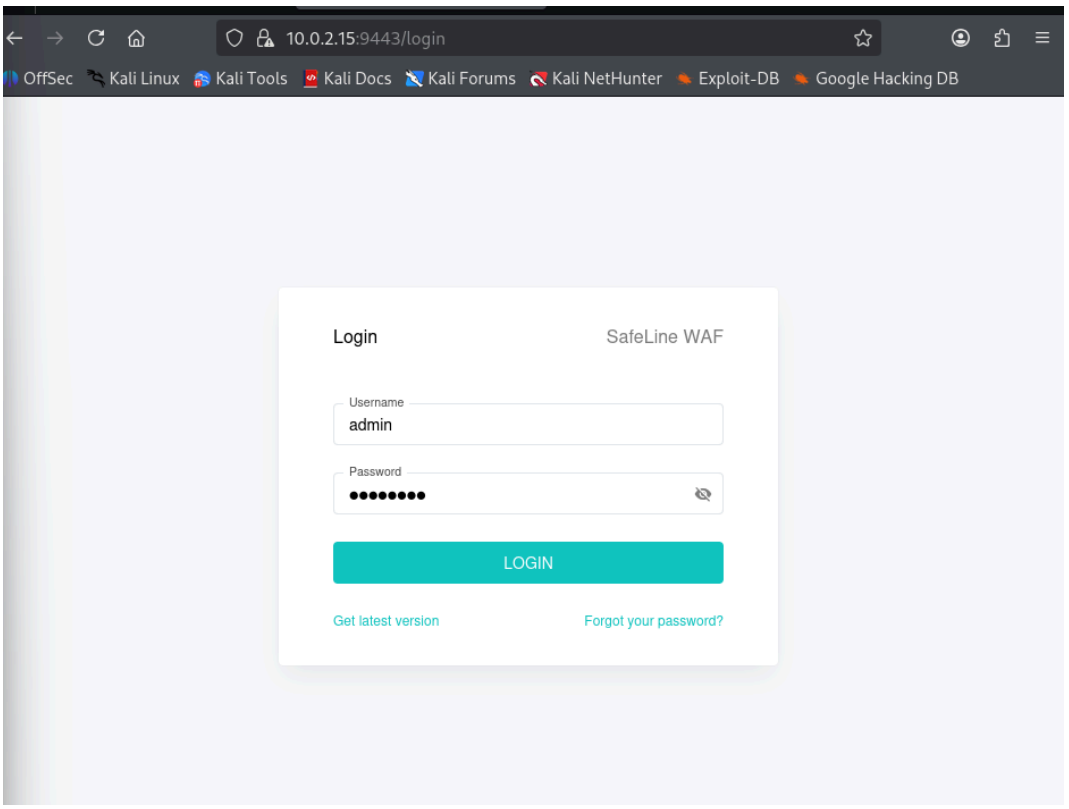
- Installing as a **Docker container** on Ubuntu
- Setting up an **Nginx-based reverse proxy** that will sit in front of DVWA
- Opening port **9443** for the admin UI
- Opening port **80/443** to receive traffic and forward it to DVWA on 8080

Step 10 - Log into SafeLine

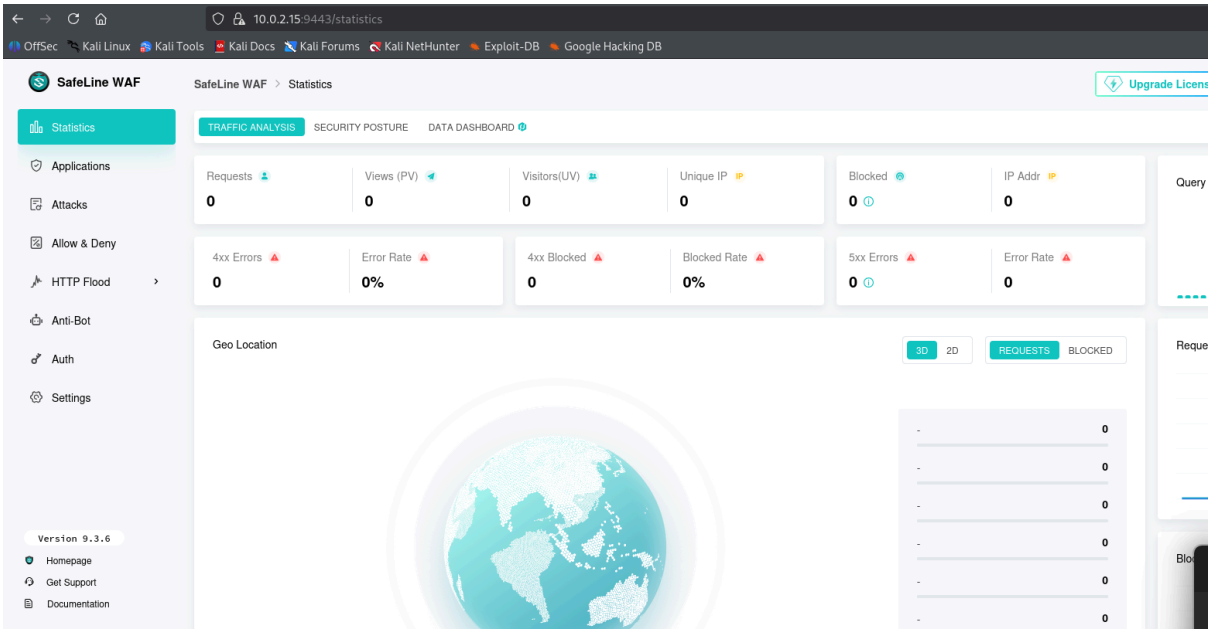
From Kali browser, go to:

https://10.0.2.15:9443

You'll see a certificate warning that's normal. Click: Advanced → Accept the Risk and Continue You'll see the SafeLine login page



The dashboard;



Step 11 - Import SSL Certificate

In the SafeLine dashboard on **Kali browser**:

1. Go to settings then look for **Certificates** in the left menu
2. Click **Add** or **Upload**
3. Upload the two files we created earlier:
    - **Certificate file:** /etc/ssl/dvwa/dvwa.crt
    - **Key file:** /etc/ssl/dvwa/dvwa.key

But first we need to get those files content. On **Ubuntu** run:

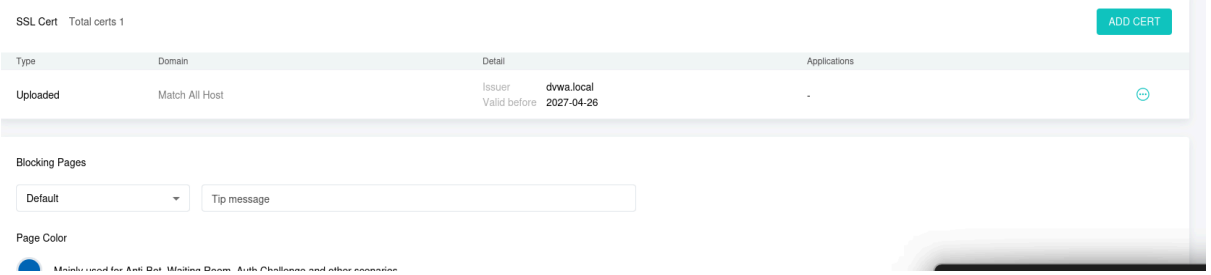
sudo cat /etc/ssl/dvwa/dvwa.crt

And

sudo cat /etc/ssl/dvwa/dvwa.key

```
halima@halima-VirtualBox:~$ sudo cat /etc/ssl/dvwa/dvwa.crt
[sudo] password for halima:
-----BEGIN CERTIFICATE-----
MIID/TCCAuWgAwIBAgIUZwlgY9XWoT1tWwc7XydyrAkMKFUwDQYJKoZIhvcNAQEL
BQAwgY0xCzAJBgNVBAYTAklEMRAwDgYDVQQIDAdKYWthcnRhMRAwDgYDVQQHDAdK
YXJrYXRhMREwDwYDVQQKDAhEVldBIExhYjERMA8GA1UECwwIU2VjdXJpdHkxEzAR
BgNVBAMMCmR2d2EubG9jYWwxHzAdBgkqhkiG9w0BCQEWEGFkbWluQGR2d2EubG9j
YWwwHhcNMjYwNDI2MTUyMTUxWhcNMjcwNDI2MTUyMTUxWjCBjTELMAkGA1UEBhMC
SUQxEDAOBgNVBAgMB0pha2FydGExEDAOBgNVBAcMB0phcmthdGExETAPBgNVBAoM
CERWV0EgTGFiMREwDwYDVQQLDAhTZWN1cml0eTETMBEGA1UEAwwKZHZ3YS5sb2Nh
bDEfMB0GCSqGSIb3DQEJARYQYWRtaW5AZHZ3YS5sb2NhbDCCASIwDQYJKoZIhvcN
AQEBBQADggEPADCCAQoCggEBAJcWGP+Z7MGT9JrVGIt5iW1l5tdbOkTMdYk+gnFv
kLYwS8fhz2b5pCqvGSQAGaw80LAv3tZTTJhdtLLS+lpUSTwdMNgqJ36WWl1bkINN
BPH6S0OcjEyVWanjKoDmJEzQ0arvOo0tsThuhVUnboRA6+p9hZLZu830UHiFoLxl
uKrhaX0LFDhya8P+Zaa2zXgP3tEZEI2cv+wscjDEH9IKgzluMkAgYiG2Ek2+xOf/
Gpj9rpEtp/aaJzEQpYszKxjr9S1XPy/dyLYX4LpQJmritMZLZv801/8Cu1g2xEjO
86wvTTz/16g/o1BEpGJfm9EyUwxmiRYQt6Cr16NkXceR+wkCAwEAAaNTMFEwHQYD
VR0OBBYEFA1hRViY19p7PqqeshHSwBhVF2UuMB8GA1UdIwQYMBaAFA1hRViY19p7
PqqeshHSwBhVF2UuMA8GA1UdEwEB/wQFMAMBAf8wDQYJKoZIhvcNAQELBQADggEB
AG1k5xSXbcYbJlwraCikHfSEVKomqkxm9gOIgjZWZjZAD846QbgoHc6p9cT6CMDy
d6HPiLiufTGdfNaY5hWpVTaB9qflPukQwy66czRdnx4hqJmCUbayX3fFvZSNeRfk
IJTAlo9uvG4IRDsnvUD5HfK83jmzEiOWYYkAJmHsGo4SZpsY7BT5cLEldzngUfzL
Zoq9pSCiV70Ui9wEgknoeGEAYGj7jltpGZcxKHyiV5xLCNCNk2IzqQHNfl8dHE+i
WVIvpQQqCoSsUO71NbJqSxCLHKiKazcTcsCfRftiKF7XexfipcmJbPxns9L3d+bt
YYahwKuh4qgzZOrAI+GHDRk=
-----END CERTIFICATE-----
halima@halima-VirtualBox:~$ sudo cat /etc/ssl/dvwa/dvwa.key
-----BEGIN PRIVATE KEY-----
MIIEvAIBADANBgkqhkiG9w0BAQEFAASCBKYwggSiAgEAAoIBAQCXFhj/mezBk/Sa
1RiLeYltZebXWzpEzHWJPoJxb5C2MEvH4c9m+aQqrxkkABmsPNCwL97WU0yYXbSy
0vpaVEk8HTDYKid+llpdW5CDTQTx+ktDnIxMlVmp4yqA5iRM0NGq7zqNLbE4boVV
J26EQOvqfYWS2bvN9FB4haC8Zbiq4Wl9CxQ4cmvD/mWmts14D97RGRCNnL/sLHIw
xB/SCoM5bjJAIGIhthJNvsTn/xqY/a6RLaf2micxEKWLMysY6/UtVz8v3ci2F+C6
UCZq4rTGS2b/NNf/ArtYNsRIzvOsL008/9eoP6NQRKRiX5vRMlMMZokWELegq9ej
ZF3HkfsJAgMBAAECggEADDWAWHqn375fnuRsoEFhFNqjdGXTmAF9V6rEupBv/VJP
9At1abDvg8spqQJWEML1qoslGCTb6hhxFJ3zwPlnKp5W7FzpsJ3LQxiLXW+LoFc1
c6BSqRH3kIDXgNo4IY9NUMwWAJhqO1vDrALXTYTRMPOAPVfuyHY/S3dMQB/TNAG1
Hhsgr8Hvx5YSs7Skpr+w/uS9NIJsvUBtUbqs0bfceNQPJoYstm+VzLTdCe7ZsO82
72uAPpypGseh9AA7t14etYjEzxHrabrGChaKkrQR4wp9Dqv1D6ajpVF9snz9YHiS
Ef0UNk6/UMZjGbrKXp4GUeWZhgJBW090NWsHFWfoRQKBgQC4s2vpw5G8b9GX0ANC
```

In SafeLine you will paste the content in the text box



Step 12 - Onboard DVWA into SafeLine

Now we tell SafeLine to protect DVWA. In the SafeLine dashboard on **Kali browser**:

- Look for Applications in the left sidebar and add application

You'll see a form. Fill it in like this:

Domain field

- First type dvwa.local → press Enter
- Then type www.dvwa.local → press Enter
- The * wildcard should already be there

Port: 443 with **HTTPS**

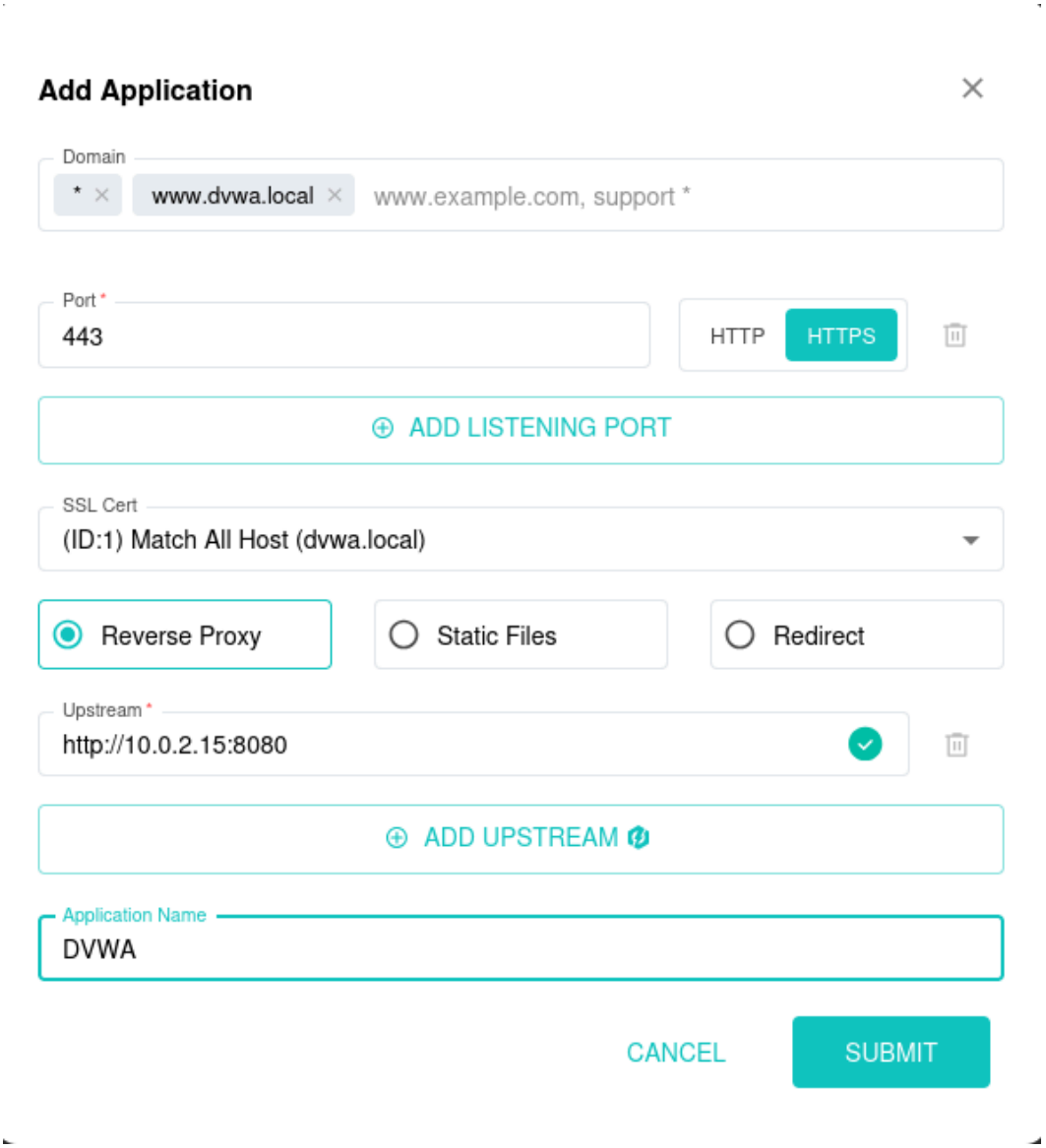
SSL Cert: (ID:1) Match All Host (dvwa.local)

Reverse Proxy: selected

Upstream: http://10.0.2.15:8080

Application Name: DVWA

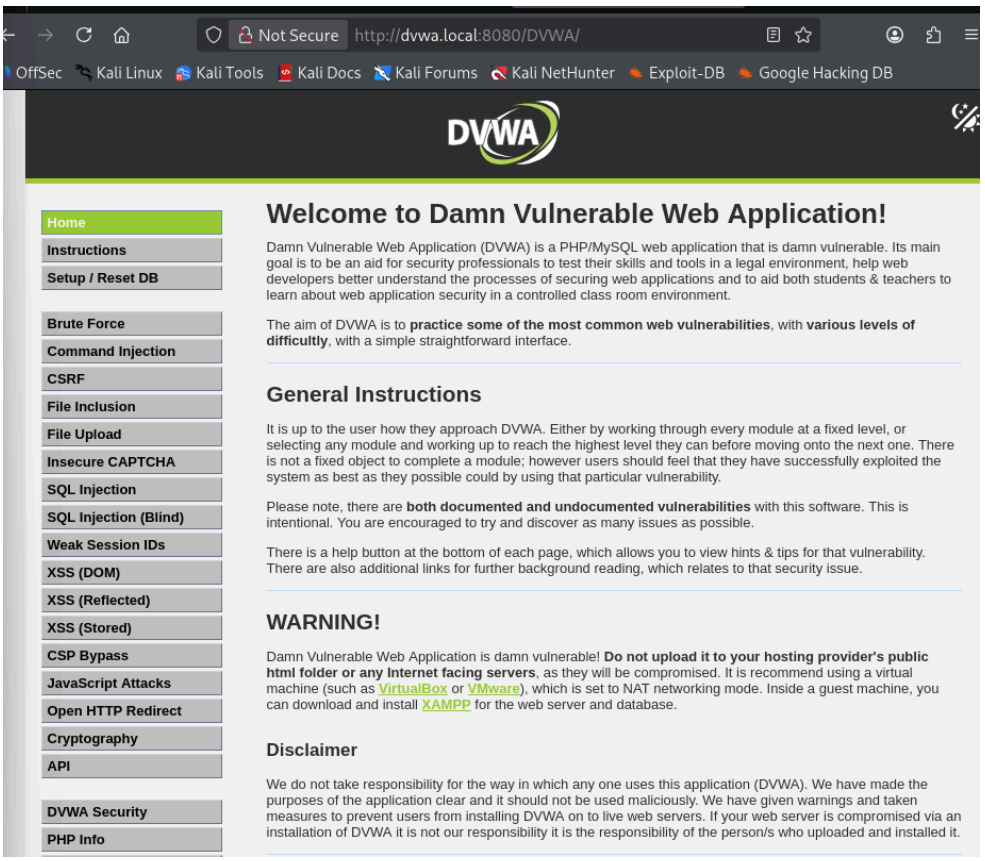
Then click **SUBMIT**.



Then from **Kali browser** go to:

https://dvwa.local/DVWA

Where you'll see this



Now log in to DVWA and once logged in:

1. Click **DVWA Security** in the left menu
2. Set security level to **Low**
3. Click **Submit**

4. Attack Demonstrations

4.1 SQL Injection Attack

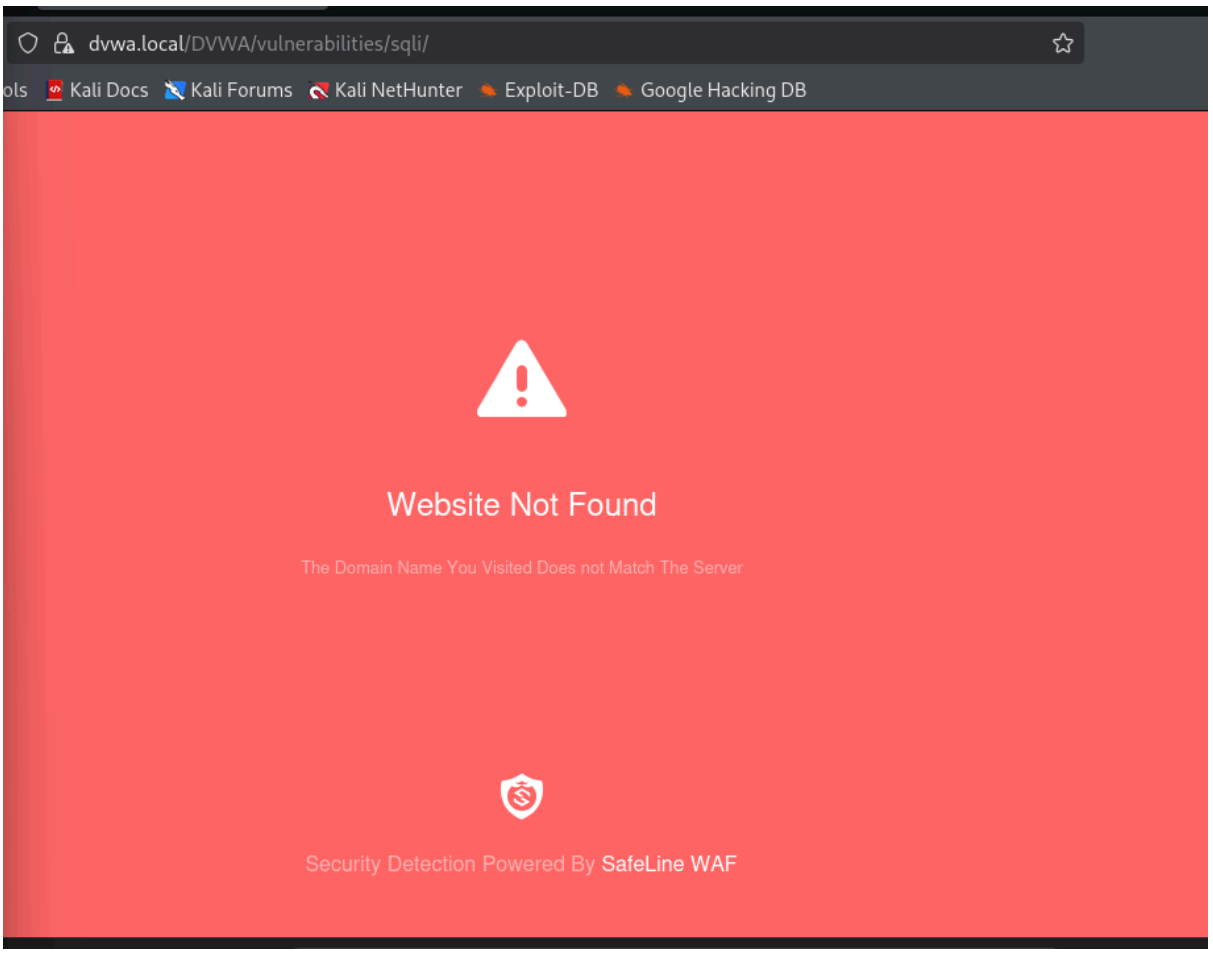
Once logged in and security set to Low:

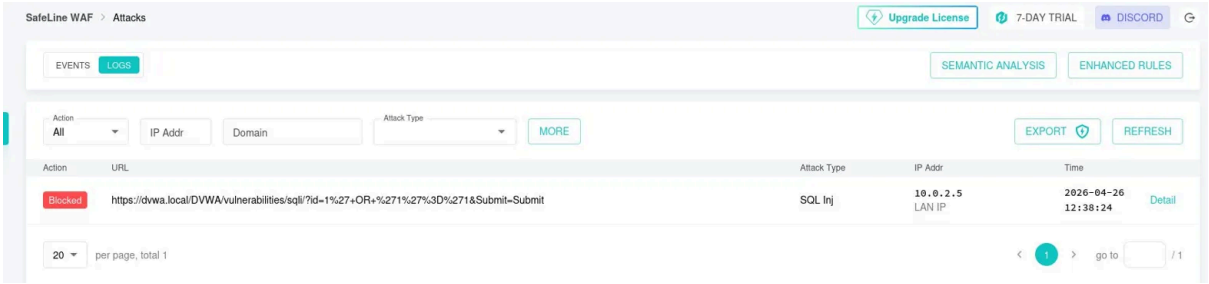
1. Click **SQL Injection** in the left menu
2. In the User ID box type:

1' OR '1'='1

3. Click **Submit**

Then go to **SafeLine dashboard** at https://10.0.2.15:9443 and check the **logs/events**; you should see the attack was **detected and blocked**!



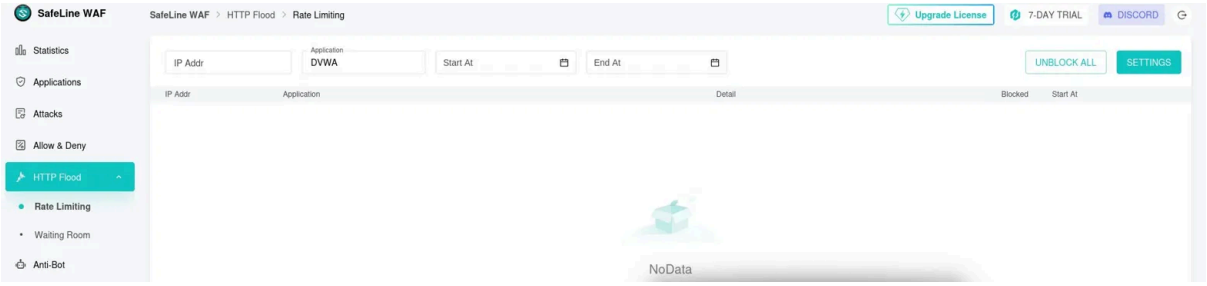


SafeLine WAF detected and BLOCKED the SQL injection from Kali

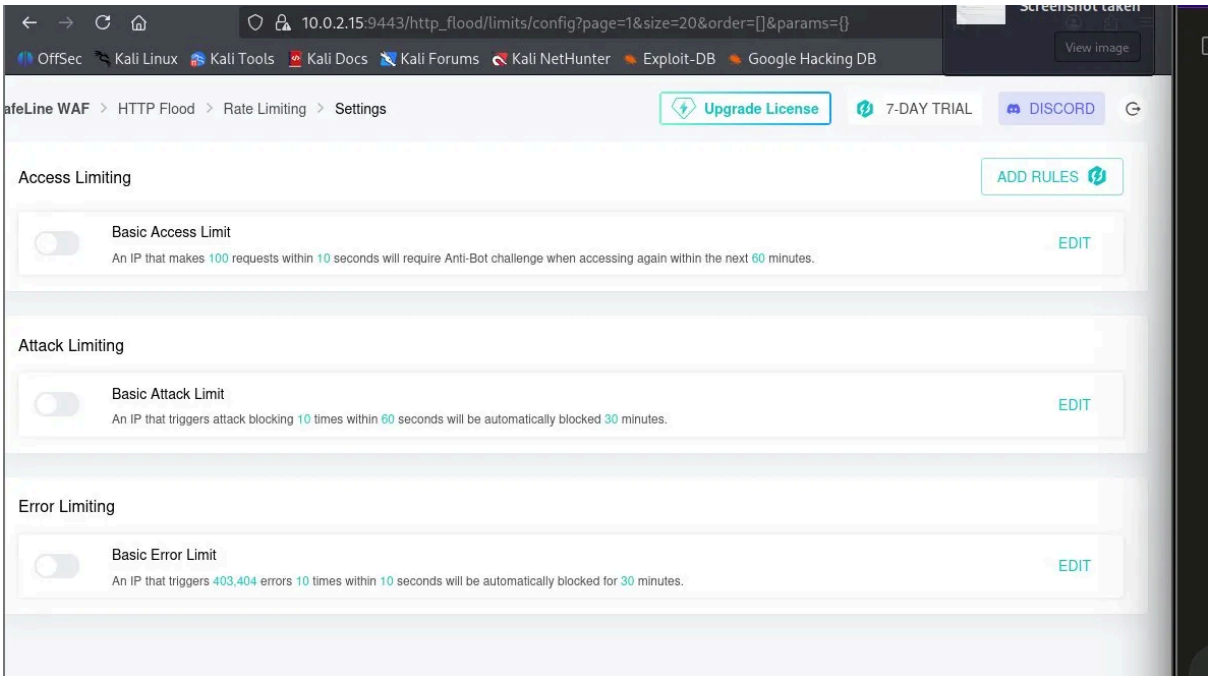
4.2 - HTTP Flood Defense

This protects against DoS attacks where Kali sends hundreds of requests per second to overwhelm the server.

In **SafeLine dashboard** on Kali browser, click **HTTP Flood** in the left sidebar.



Click the **SETTINGS** button in the top right corner to configure the rate limiting rules.



Toggle ON all three by clicking each switch:

1. Click the toggle next to **Basic Access Limit**
2. Click the toggle next to **Basic Attack Limit**
3. Click the toggle next to **Basic Error Limit**

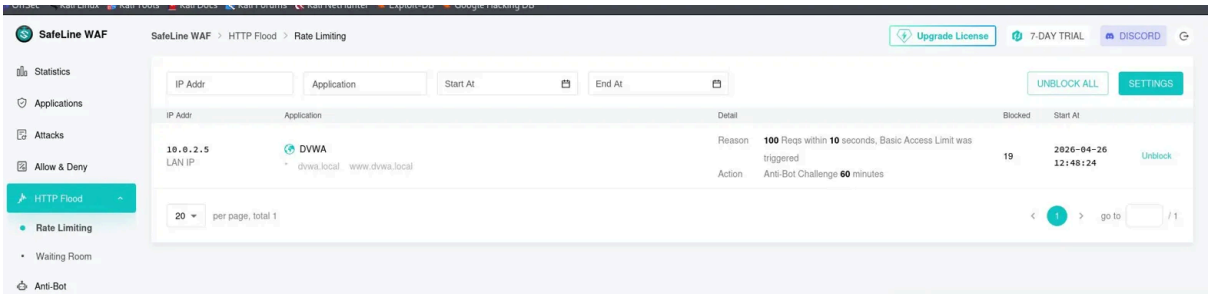
Then let's **test it** from **Kali terminal**. Run this to flood DVWA with requests

for i in {1..200}; do curl -k https://dvwa.local/ ; done

HTTP Flood Defense is WORKING. Look at the Rate Limiting page:

Kali got blocked for flooding DVWA with too many requests.

And in the Kali terminal you can see SafeLine served the **Anti-Bot Challenge page** instead of DVWA.



4.3 - Block Kali's IP Completely

Click the BLACKLIST tab, then click the ADD **RULES** button in the top right.

Fill in the rule like this:

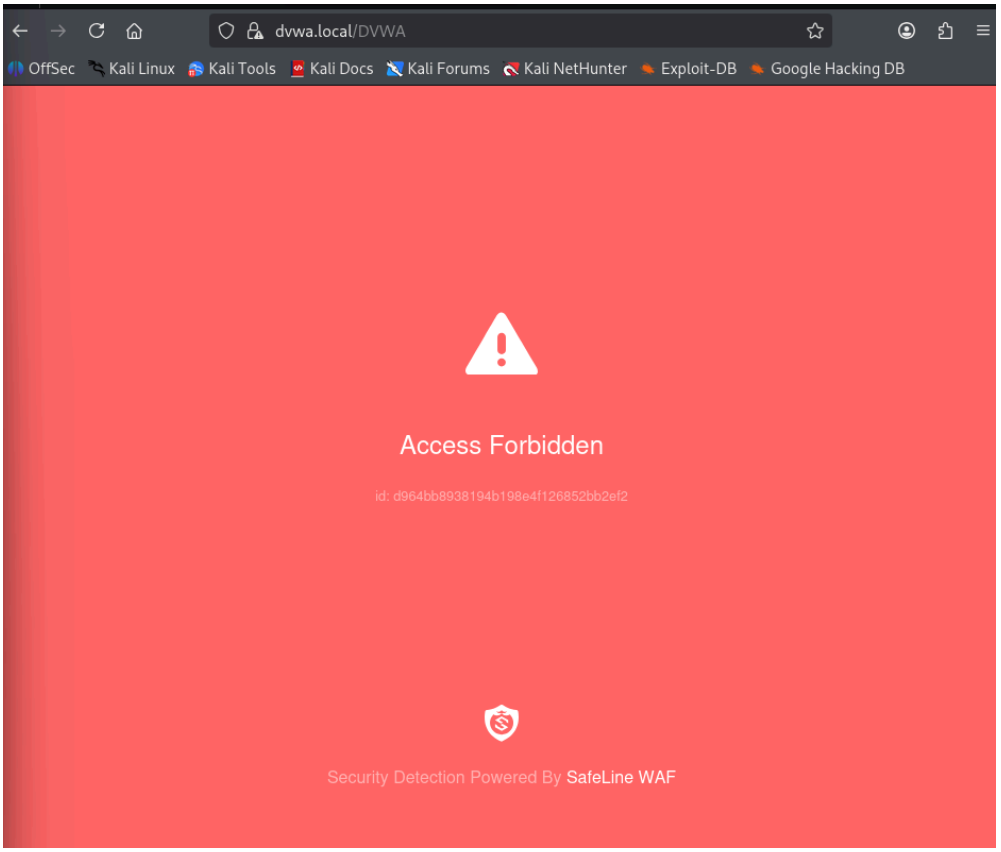
- **Name:** Block Kali
- **Type:** Source IP
- **Value:** 10.0.2.5
- **Action:** Deny

Click **Save/Submit**.

Now let's **test it**! From **Kali browser** try to visit:

https://dvwa.local/DVWA

SafeLine WAF is fully working - Kali is completely blocked! It says forbidden



4.4 - Auth Gateway

This forces anyone visiting DVWA to authenticate through SafeLine first before even reaching the site.

In SafeLine dashboard go to **Applications → DVWA → AUTH** button.

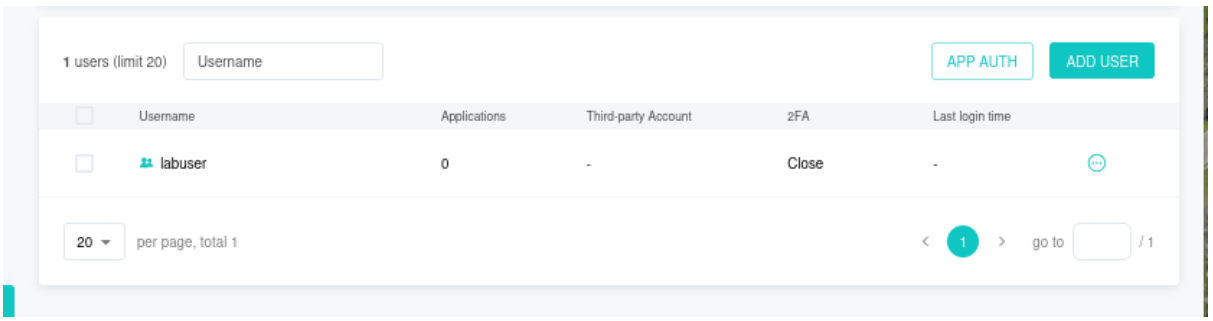
Click **SETTINGS** button in the top right to configure it!

Step 1 - Add a User

Click **ADD USER** button and fill in:

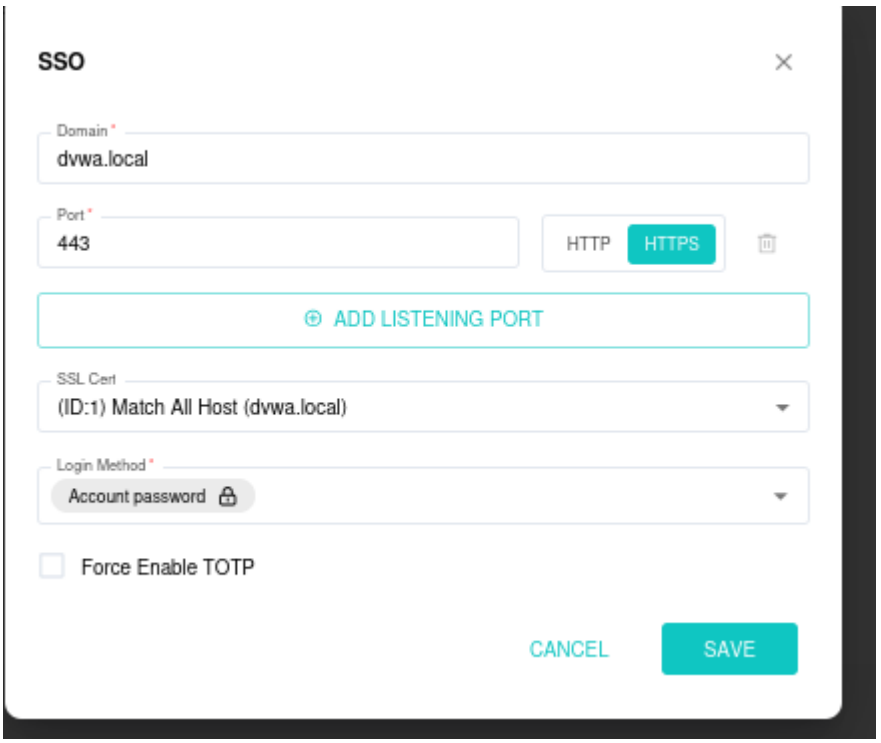
- **Username:** labuser
- **Password:** Lab@12345

Click Save.

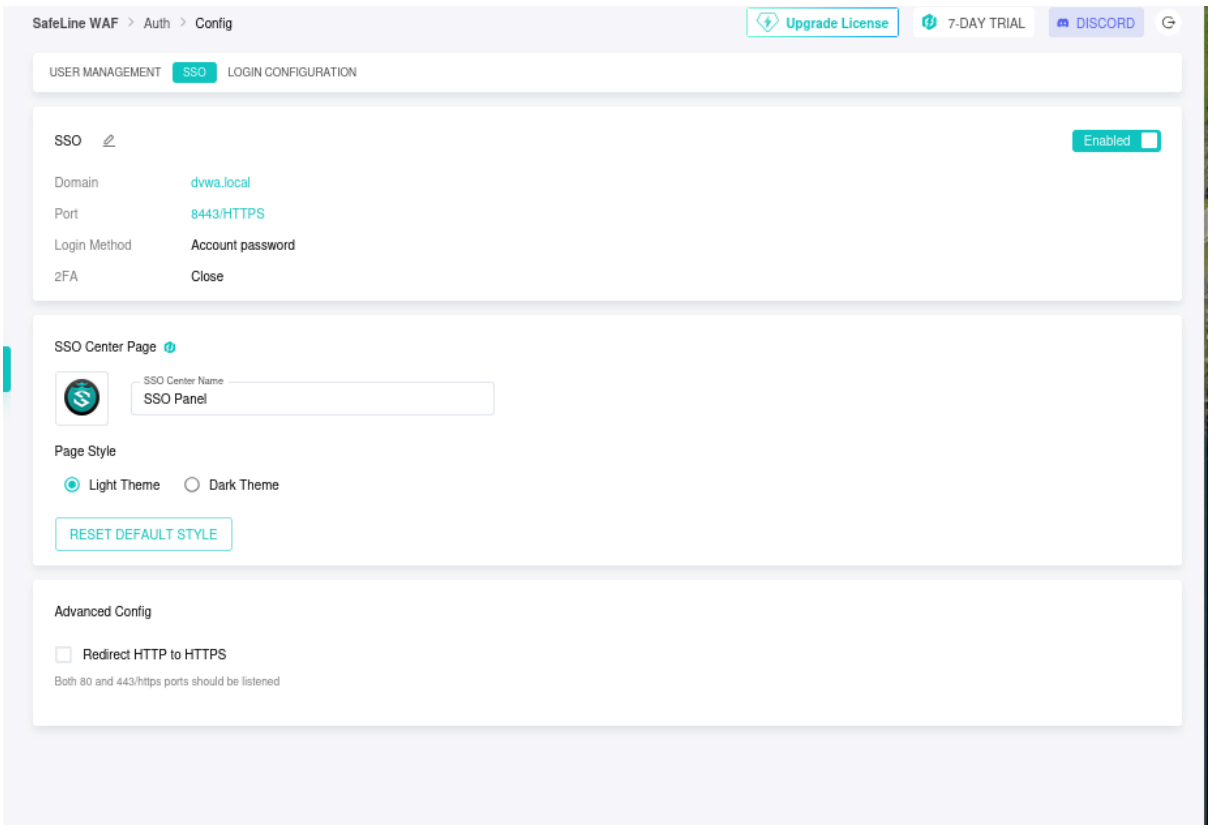


To make this work I did the following:

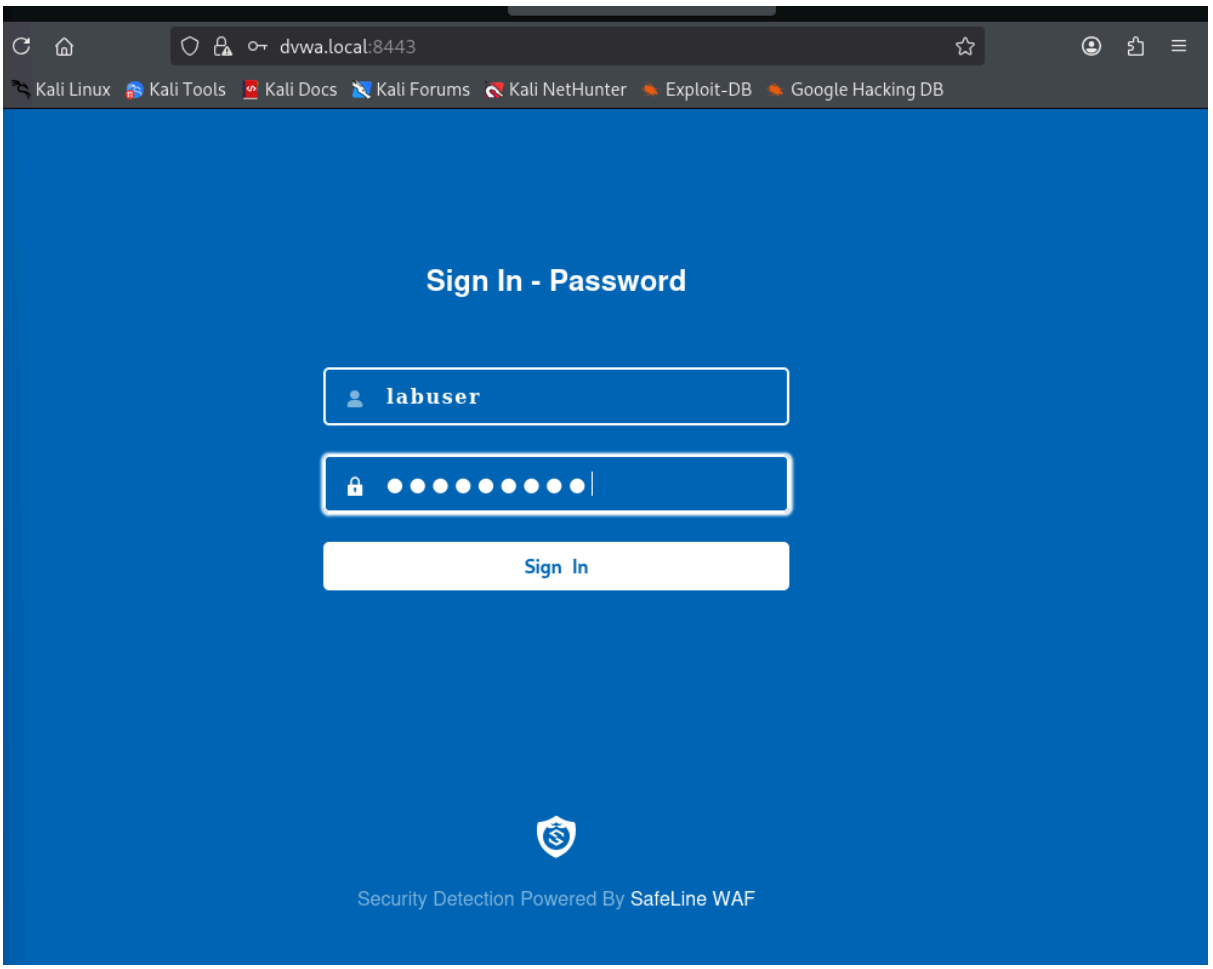
1. Enabled Auth Gateway (SSO) on SafeLine for the DVWA application
2. Created a user labuser with password Lab@12345
3. Configured it on port 8443 with HTTPS

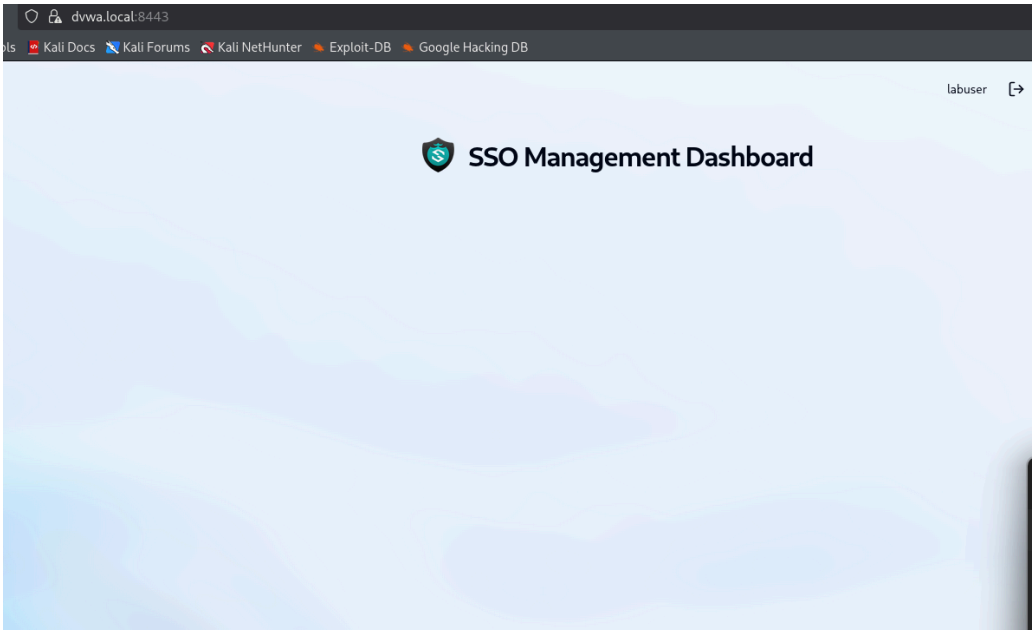


Go to SafeLine → AUTH SETTINGS → USER MANAGEMENT → edit labuser → enable DVWA under App Auth → save.



4. Successfully logged in through SafeLine's SSO portal at https://dvwa.local:8443





What it proves:

Anyone trying to reach DVWA must first authenticate through SafeLine's Auth Gateway - adding an extra layer of protection in front of the vulnerable app.

4.5 - XSS Attack

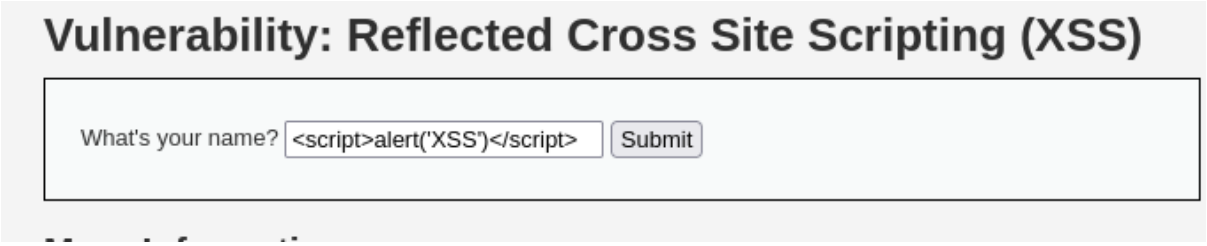
Try a Cross-Site Scripting attack from Kali and see SafeLine block it

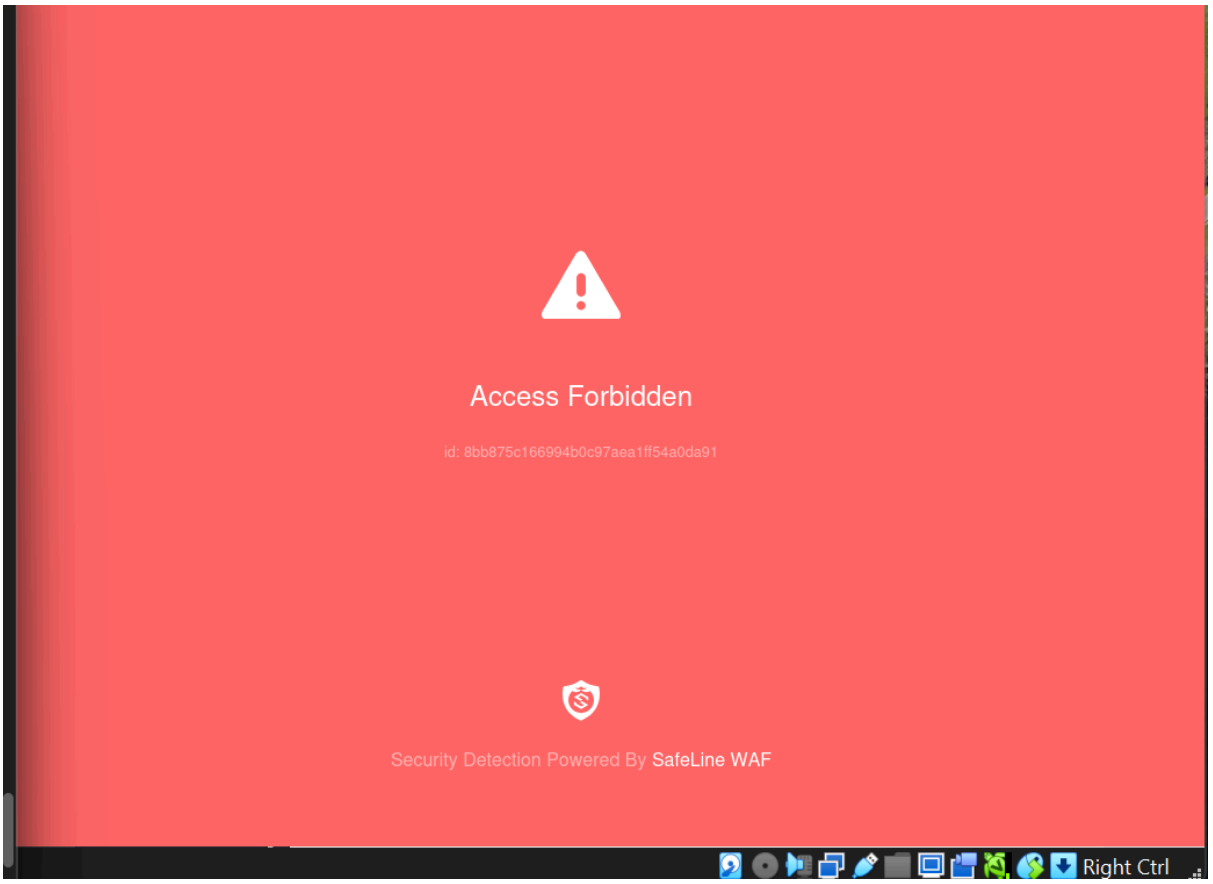
Let's start with XSS. On Kali browser go to:

https://dvwa.local/DVWA/vulnerabilities/xss_r/

Make sure DVWA security is set to **Low**, then in the input box type:

<script>alert('XSS')</script>





SafeLine dashboard → Events/Attacks — shows the event being blocked.

Action	URL	Attack Type	IP Addr	Time	
Blocked	https://dvwa.local/DVWA/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27XSS%...	XSS	10.0.2.5 LAN IP	2026-04-26 13:36:51	Detail

4.6 - Command Injection

Try a command injection attack and see SafeLine block it

On Kali browser go to:

https://dvwa.local/DVWA/vulnerabilities/exec/

Make sure DVWA security is still set to **Low**, then in the IP address box type:

; ls -la /etc

Vulnerability: Command Injection

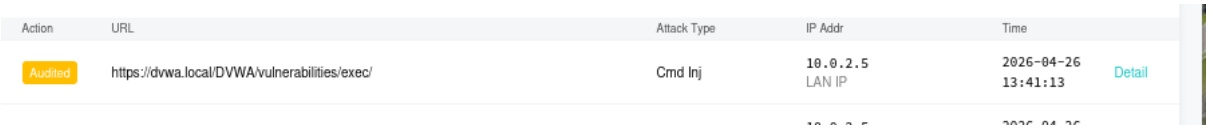
Ping a device

Enter an IP address: [ ] Submit

```
total 1160
drwxr-xr-x 135 root root   12288 Apr 26 11:31 .
drwxr-xr-x  21 root root    4096 Apr 26 11:31 ..
-rw-------   1 root root       0 Aug  7  2023 .pwd.lock
drwxr-xr-x   4 root root    4096 Aug  7  2023 ModemManager
drwxr-xr-x   7 root root    4096 Apr 26 10:18 NetworkManager
drwxr-xr-x   2 root root    4096 Apr 26 10:18 PackageKit
drwxr-xr-x   2 root root    4096 Aug  7  2023 UPower
drwxr-xr-x  12 root root    4096 Aug  7  2023 X11
drwxr-xr-x   3 root root    4096 Aug  7  2023 acpi
-rw-r--r--   1 root root    3028 Aug  7  2023 adduser.conf
drwxr-xr-x   3 root root    4096 Aug  7  2023 alsa
drwxr-xr-x   2 root root    4096 Apr 26 10:42 alternatives
-rw-r--r--   1 root root     335 Mar 23  2022 anacrontab
drwxr-xr-x   8 root root    4096 Apr 26 11:01 apache2
-rw-r--r--   1 root root     433 Mar 23  2022 apg.conf
drwxr-xr-x   5 root root    4096 Aug  7  2023 apm
drwxr-xr-x   3 root root    4096 Apr 26 10:17 apparmor
drwxr-xr-x   8 root root    4096 Apr 26 10:42 apparmor.d
drwxr-xr-x   3 root root    4096 Apr 26 10:18 apport
-rw-r--r--   1 root root     769 Feb 22  2022 appstream.conf
drwxr-xr-x   8 root root    4096 Mar 22 05:37 apt
drwxr-xr-x   3 root root    4096 Apr 26 10:18 avahi
-rw-r--r--   1 root root    2319 Jan  6  2022 bash.bashrc
-rw-r--r--   1 root root      45 Nov 11  2021 bash_completion
drwxr-xr-x   2 root root    4096 Apr 26 10:42 bash_completion.d
-rw-r--r--   1 root root     367 Dec 16  2020 bindresvport.blacklist
drwxr-xr-x   2 root root    4096 Apr  7  2022 binfmt.d
drwxr-xr-x   2 root root    4096 Apr 26 10:17 bluetooth
-rw-r-----   1 root root      33 Aug  7  2023 brlapi.key
drwxr-xr-x   7 root root    4096 Aug  7  2023 brltty
-rw-r--r--   1 root root   29219 Jun 28  2022 brltty.conf
drwxr-xr-x   3 root root    4096 Aug  7  2023 ca-certificates
```

This tries to inject a system command alongside a legitimate ping request. SafeLine should detect and block it.

SafeLine dashboard → Events/Attacks — shows the command injection attempt logged!



Action	URL	Attack Type	IP Addr	Time	
Audited	https://dvwa.local/DVWA/vulnerabilities/exec/	Cmd Inj	10.0.2.5 LAN IP	2026-04-26 13:41:13	Detail

5. Lab Summary

The following table summarises all components successfully configured and tested in this lab:

Component	Status / Details
NAT Network Setup	Both VMs on 10.0.2.0/24, internet accessible, DVWA isolated
LAMP Stack	Apache2 (port 8080), PHP 8.1, MySQL 8.0 — all running
DVWA	Installed, configured, database initialised with test data
DNS Resolution	dvwa.local resolving to 10.0.2.15 on both VMs
SSL Certificate	Self-signed cert for dvwa.local (365 days)
SafeLine WAF	v9.3.6 installed via Docker, admin UI on port 9443
SQL Injection	Detected and BLOCKED by SafeLine WAF
XSS Attack	Detected and BLOCKED by SafeLine WAF
Command Injection	Detected and BLOCKED by SafeLine WAF
HTTP Flood Defense	Rate limiting triggered at 100 req/10s, Kali blocked
Custom IP Block	Kali IP 10.0.2.5 completely blocked via blacklist rule
Auth Gateway (SSO)	Pre-authentication enforced on port 8443 via SafeLine SSO

6. Conclusion

This lab successfully demonstrated the setup and operation of a Web Application Firewall home lab using SafeLine WAF. Starting from bare virtual machines, a complete attack-and-defend environment was built, tested, and documented.

SafeLine WAF proved effective at detecting and blocking multiple attack types including SQL injection, Cross-Site Scripting, and command injection. Its additional features such as HTTP flood rate limiting, IP-based blacklisting, and SSO authentication gateway provide multiple layers of defence that go beyond basic attack detection.

Key takeaways from this lab:

- A WAF operates as a reverse proxy, intercepting traffic before it reaches the application
- Defence-in-depth is critical — SafeLine's Auth Gateway, rate limiting, and blacklisting work together to protect applications
- NAT Network is a safer choice than Bridged networking for labs involving intentionally vulnerable applications
- Self-signed certificates enable HTTPS in isolated lab environments without requiring a public CA

Potential next steps for further exploration include integrating a SIEM for centralised log analysis, adding additional vulnerable applications like OWASP Juice Shop, exploring WAF bypass techniques at higher DVWA security levels, and integrating IDS/IPS tools alongside the WAF.

End of Report